EffiMatch: Enabling Fast and Accurate Learning-based Packet Classification

Lida Liao[1,4], Yuhan Wu[1,4], Jiashuo Yu[1,4], Longlong Zhu[3], Hongyan Liu[1,4], Junjie Lin[1,4], Dong Zhang[1,2,4], Xiang Chen[3], Chunming Wu[3]
[1]College of Computer and Data Science, Fuzhou University, [2]Zhicheng College, Fuzhou University, [3]Zhejiang University, [4]Fujian Province Key Laboratory of Information Security of Network System, Fuzhou University

2025 IEEE 33rd International Conference on Network Protocols (ICNP) | 979-8-3315-0376-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICNP65844.2025.11192436

***Abstract*—Learning-based Packet Classification methods reduce memory overhead by using lightweight Recursive Model Index(RMI) structures to limit the search range, followed by linear matching. However, they face a trade-off: complex RMI structures achieve smaller search ranges but slow down lookup, while simpler ones are faster but require larger scans. In this paper, we propose EffiMatch, a parallel multi-model lookup architecture aimed at resolving the trade-off between RMI complexity and linear search range in learning-based index systems. We propose two key designs: 1) We design a partitioning strategy called Distribution-Distance Partitioning (DDP), which groups data points with similar trends into the same segment. Combined with parallel lookup, this reduces the linear search range while maintaining high lookup speed. 2) We propose a more fine-grained binarization method, Base-Index Representation (BI), which approximates floating-point operations using integers. This method further reduces the search range without increasing model complexity. Experimental results show that EffiMatch reduces the linear search range by 26.84% using lower-complexity RMI models, which improves lookup speed by up to 6× and reduces construction time by up to 4 orders of magnitude compared to state-of-the-art LPC methods.**

I. Introduction

Packet classification is essential in network devices for functions like access control, firewalls, and QoS [1–3]. It involves matching incoming packets to multifield rules and executing actions. As network scale grows, modern data centers require classifiers to support millions of rules [4], with lookup speed and rule capacity becoming critical factors for performance [5–7]. In service function chain (SFC) scenarios, devices such as network switches and SmartNICs [8] must rapidly determine the processing path for each packet, imposing strict demands on classification speed and accuracy [9]. However, these devices often have limited on-chip memory, typically just a few hundred kilobytes of SRAM [10], making fast, accurate classification a significant challenge.

Learning-based data structures [11] have recently been applied to packet classification to manage large rule sets under limited hardware resources. These methods train models to predict the approximate position of the matched rule, and then perform a serial search within a narrowed range to find the exact match. This approach reduces memory overhead by avoiding the need to store the entire rule table. Learning-based Packet Classification (LPC) [12–15] typically adopts a recursive model index (RMI), where each submodel handles a subspace of the input, and models are hierarchically composed to guide lookup. The final segment is searched linearly. This approach supports large-scale rule sets (e.g., one million rules) using compact models as small as 35KB, enabling efficient packet classification. Current LPC methods generally follow two directions. One is represented by NuevoMatch [14], which adopts deep neural architectures to build complex RMI models, aiming to improve prediction accuracy and minimize the search range. The other is exemplified by NeuTree [15], which constructs simplified RMI models through model binarization and rule partitioning, enabling hardware deployment while achieving high-throughput lookups.

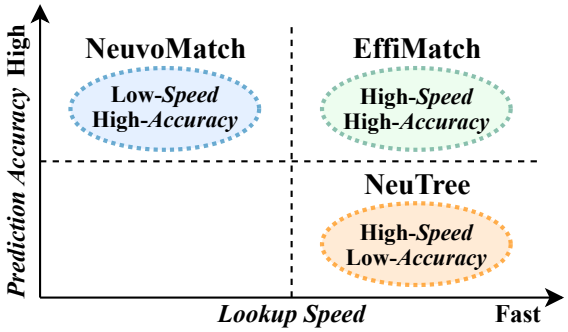


Fig. 1: RMI Complexity and Linear Search Range.

However, existing learning-based packet classification methods struggle to balance the trade-off between the complexity of the RMI structure and the size of the linear search range. NuevoMatch [14] adopts deep neural networks to build complex RMI models, enabling accurate rule position prediction and reduced search range. However, the reliance on floating-point operations prevents hardware deployment on platforms such as Tofino [16], limiting its practical lookup performance. In contrast, NeuTree [15] uses lightweight binarized linear models [17] to construct simplified RMIs, eliminating floating-point operations and supporting hardware deployment with higher throughput. Yet, these models offer limited expressiveness, resulting in larger search ranges. We argue that this trade-off between RMI complexity and linear search range is not unavoidable. As real-time traffic monitoring becomes increasingly critical, classification systems must achieve both fast response and accurate rule matching [18–20]. However, existing LPC methods still fall short of meeting these dual

Lida Liao and Yuhan Wu contributed equally to this work.

979-8-3315-0376-5/25/$31.00 © 2025 IEEE

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

demands simultaneously.

In this paper, we propose EffiMatch, a learning-based index architecture that supports multi-model parallel lookup, with the goal of balancing the complexity of the RMI structure and the size of the linear search range in packet classification. Our core idea is to partition the global rule set into multiple segments, assign a deployable linear model to each segment, and perform local fitting and lookup independently, with all models running in parallel. By optimizing both the rule distribution in preprocessing and model granularity during construction, EffiMatch reduces search range without increasing model complexity, enabling fast and accurate classification. As illustrated in Figure 1, EffiMatch breaks the trade-off between RMI complexity and linear search range, achieving a performance balance superior to prior works such as NeuTree [15] and NuevoMatch [14]. Therefore, the EffiMatch architecture is designed to achieve the following two goals.

1) The trade-off in the rule preprocessing stage. In real-world rule sets, the mapping between keys and rule positions often shows highly uneven patterns, including abrupt slope changes and local nonlinearities. Existing LPC methods typically use fixed interval partitioning, which fails to separate regions with distinct trends. This results in poor linear fitting and a larger linear search range.

In response, we propose Distribution-Distance Partitioning (DDP), which iteratively fits lines to key-position pairs and groups points with similar trends into the same segment based on residual thresholds. This effectively reduces the linear search range. To accelerate preprocessing, we also design a parallel segmentation strategy that constructs thousands of segments within milliseconds, maintaining high preprocessing efficiency while ensuring fine-grained partitioning.

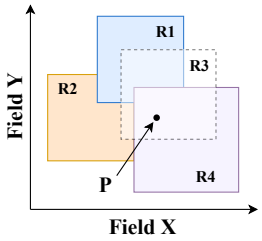
2) The trade-off in the model construction stage. Floating-point neural networks represent complex RMI models that achieve smaller search ranges but are computationally expensive and unsuitable for hardware deployment. In contrast, traditional binarized models correspond to simple RMI structures that support fast inference and hardware deployment, but their limited expressiveness leads to larger search ranges.

In response, we propose Base-Index Representation (BI), which replaces floating-point weights with an expressive integer-only format. Each weight is encoded as $\text{base} \times 2^{\text{index}}$, preserving value granularity while eliminating the need for floating-point operations. Compared to deep neural models like NuevoMatch [14] that rely on complex RMI structures, BI uses lower-complexity RMI models to achieve a comparable search range, while reducing model construction time by up to 4 orders of magnitude.

Furthermore, we implement a software prototype of the EffiMatch multi-model lookup architecture on a software platform, covering the complete process from the training phase to the query phase. The system demonstrates good lookup performance on real-world rule sets.

In summary, this paper makes the following contributions:

- We propose EffiMatch, a multi-model parallel lookup architecture for learning-based packet classification, which achieves a balanced trade-off between RMI complexity and linear search range for rule matching (see §III-A).
- We design a Distribution-Distance Partitioning strategy (see §III-B) to reduce the linear search range via trend-based segmentation and parallel lookup. We also propose a Base-Index Representation model (see §III-C) that approximates floating-point precision using integers, further reducing the search range without increasing model complexity.
- We evaluate EffiMatch on the ClassBench benchmark. It reduces the linear search range by 26.84% with lower-complexity RMI models, improves lookup speed by up to 6×, and shortens construction time by up to 4 orders of magnitude (see §IV).



Rule#	Field X	Y	Priority	Action
R1	100*	1***	1	Forward
R2	01**	01**	0	Drop
R3	11**	1***	8	Forward
R4	10**	0***	5	Modify

Fig. 2: An example of the rule.

II. BACKGROUND AND MOTIVATION

A. Packet Classification

Packet classification can be abstracted as a point-location problem in high-dimensional space, where each packet is viewed as a point and each rule is represented as a hypercube. The classifier's task is to determine which hypercubes contain the point. Due to potential overlaps among rules, multiple hypercubes may match a packet simultaneously. Therefore, the classifier must additionally select the rule with the highest priority to finalize the matching decision. As illustrated in Figure 2, each rule (R1–R4) defines a region in the two-dimensional field space, and a packet P is represented by a black point. In this example, P matches rules R1, R3, and R4 simultaneously. However, since R3 has the highest priority among them, it is chosen as the final matching rule. Traditionally, physical switching devices rely on TCAMs to perform rule matching. However, TCAMs suffer from high power consumption and significant silicon area per rule, which severely limits their scalability. Typically, TCAMs can only support several thousand rules, falling short of the needs for matching large-scale rule sets in modern networks.

To overcome this limitation, researchers have proposed various data structures for packet classification on DRAM or SRAM to improve scalability. Among them, decision trees [21–23] and Tuple Space Search (TSS) [24, 25] are two mainstream approaches: the former achieves fast lookup by recursively partitioning the high-dimensional rule space into a tree structure, while the latter groups rules by prefix length and uses hash functions for fast matching and incremental updates. More recently [26–28], hybrid approaches have been explored to combine the strengths of different data structures,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

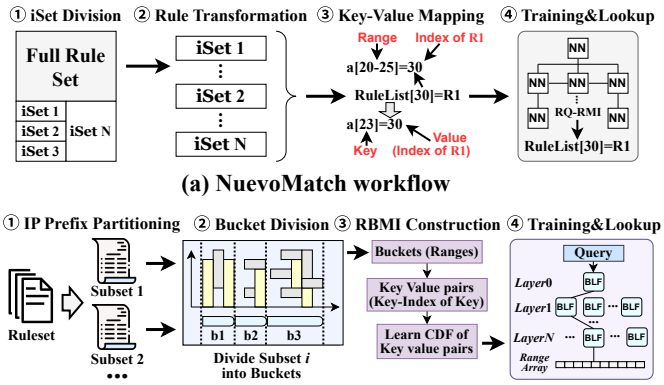


Fig. 3: Comparison of Workflow between NeuTree and NuevoMatch.

and several studies have investigated their implementation on FPGA platforms [29, 30].

B. Learning-based Packet Classification

In recent years, researchers have proposed a novel data structure named the learning-based index, which leverages machine learning models to replace traditional data structures (e.g., B-Trees and hash tables) for efficient data querying. The Recursive Model Index (RMI) [11] is a representative learning-based index structure. It employs multiple layers of recursive neural network models to predict key-value positions hierarchically from top to bottom. Specifically, the bottom-layer model outputs the final predicted position pos along with an error bound ϵ. The system then only needs to perform a linear search within the interval $[pos - \epsilon, pos + \epsilon]$ to locate the correct value. Compared to traditional data structures, RMI significantly reduces memory consumption and provides stable inference latency, making it particularly suitable for large-scale rule-query tasks.

In the domain of packet classification, learning-based index structures inspired by the Recursive Model Index (RMI) [11] have evolved into two representative lines of work. One representative line of work is NuevoMatch [14], which is primarily designed for general-purpose CPU platforms. Its key idea is to apply a range-query variant of RMI (Range Query-RMI, RQ-RMI) to construct one-dimensional rule mappings. Specifically, NuevoMatch [14] first partitions the rule set along a selected dimension into multiple non-overlapping rule subsets (iSets). Each iSet is then converted into an interval-to-rule mapping, and sampled key-value pairs are generated to train RQ-RMI neural network models, enabling efficient one-dimensional interval matching. This partition-and-mapping mechanism allows NuevoMatch to perform fast rule retrieval using a relatively small number of lightweight neural networks (521 NN models for 100k 5-tuple rules), significantly reducing overall memory consumption on CPU-based systems. As shown in Figure 3(a), NuevoMatch includes four main steps: iSet division, rule transformation, key-value mapping, and RQ-RMI-based lookup.

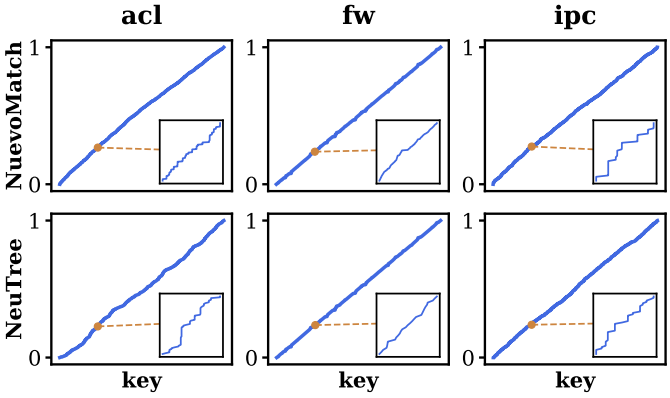


Fig. 4: Motivating Example of EffiMatch Design.

Another representative line of work is NeuTree [15], which differs from the CPU-oriented NuevoMatch [14] by adopting the Recursive Binarized Model Index (RBMI) to eliminate floating-point operations entirely. This allows efficient deployment on hardware platforms such as FPGAs [31], enabling higher query throughput. This design enables efficient deployment on hardware platforms such as FPGAs [31], where higher query throughput can be achieved. NeuTree [15] first partitions the rule set into multiple subsets based on IP prefix length, and then further divides each subset into several independent buckets. For each bucket, an RBMI is constructed, replacing conventional floating-point operations with fully binarized operations for key-to-index mapping. These RBMI sub-models can run in parallel on FPGA hardware [31], significantly accelerating rule lookup with minimal memory overhead. The full construction pipeline of NeuTree is also summarized in Figure 3(b), including prefix partitioning, bucket division, RBMI construction, and lookup execution.

C. Motivation

We observe that the key-to-rule mapping functions derived from real-world rule sets (i.e., CDF curves) often exhibit abrupt slope transitions and local irregularities (see Figure 4, e.g., acl, fw, ipc datasets). However, existing learning-based packet classification (LPC) methods typically adopt coarse-grained rule space partitioning, which fails to adapt to these complex regions, leading to increased linear model fitting error and a larger linear search range.

1) NuevoMatch: Small search range but high RMI complexity. NuevoMatch [14] uses floating-point neural networks to model complex key-to-rule mappings, enabling more accurate segmentation and reduced search range. However, these models form recursive model index (RMI) structures with high computational cost and cannot be deployed on hardware platforms such as ASICs or FPGAs, resulting in slower practical lookup speed.

2) NeuTree: Low RMI complexity but large search range. NeuTree [15] constructs low-complexity RMI structures using binarized linear models, making it suitable for deployment and fast lookup. However, it relies on coarse-grained partitioning and fixed-slope fitting, lacking adaptability to abrupt slope

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

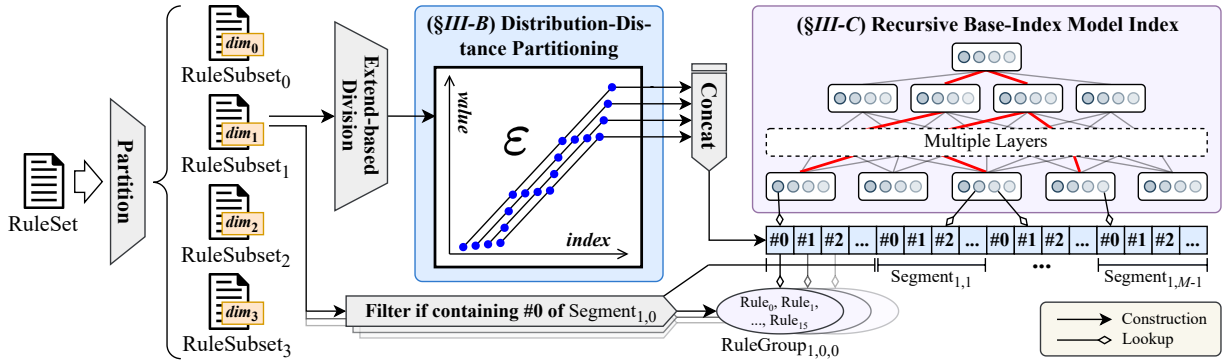


Fig. 5: EffiMatch construction workflow.

changes and local irregularities, which leads to poor local fitting and a significantly enlarged linear search range.

In summary, current LPC methods struggle to balance RMI structure complexity and linear search range. To this end, our goal is to (1) develop a distribution-aware rule partitioning strategy to reduce the range in complex regions, and (2) design low-complexity RMI models that maintain a small search range while enabling fast, integer-only lookups.

III. EffiMatch Construction Design

In this section, we will first introduce the workflow, followed by motivation and details of the two main designs.

A. Design Overview

Architecture. To achieve the goal of balancing RMI complexity and linear search range in learning-based packet classification, we propose the EffiMatch framework. As illustrated in Figure 5, EffiMatch consists of two core designs: (1) a *Distribution-Distance Partitioning (§III-B)* strategy that groups rules with similar trends into the same segment and enables parallel lookup, effectively reducing the linear search range; and (2) a multi-model structure based on the *Base-Index Representation (§III-C)*, which approximates floating-point operations using integers to further reduce the search range without increasing model complexity.

We first divide the entire rule set into four RuleSubsets, each choosing a primary query dimension to minimize rule overlap. Within each RuleSubset, EffiMatch applies DDP to segment boundary points along the chosen dimension, generating up to M local segments. For each segment, a corresponding Range-Binarized Model Index (RBIMI) is trained to support efficient region-specific prediction. During construction, each critical key value in a segment is pre-associated with a compact RuleGroup, typically containing fewer than twenty local rules. This significantly narrows the scope of subsequent matching operations. During inference, EffiMatch performs serial traversal over the four RuleSubsets and parallel lookup within each subset. Based on the query key's value, the system locates the target segment and retrieves the matched rule from its RuleGroup, achieving a balance between low RMI complexity and small search range for efficient packet classification.

Workflow. *During the training phase*, we first load the complete rule set and partition it into four RuleSubsets based on the prefix lengths of the source and destination IP fields. Specifically, we set thresholds $[t_1 = 12, t_2 = 12]$, and divide the rule set into four subsets according to whether the prefix lengths satisfy the following combinations: $\{\geq t_1, \geq t_2\}$, $\{\geq t_1, < t_2\}$, $\{< t_1, \geq t_2\}$, and $\{< t_1, < t_2\}$. For each RuleSubset_i, we select a dimension dim_i as the primary query field so that the interval overlap among all rules on this dimension is minimized. Then, for each RuleSubset_i, we apply an Extend-based division scheme similar to the one proposed in NeuTree [15], and obtain a sequence of splitting positions, denoted as SplitPosList_i. Next, we apply the Distribution-Distance Partitioning (DDP) algorithm to SplitPosList_i, dividing it into at most M Segments (from $\text{Segment}_{i,0}$ to $\text{Segment}_{i,M-1}$), and arrange these Segments sequentially in memory to support efficient parallel queries. For each $\text{Segment}_{i,j}$, we construct a corresponding lightweight rule group $\text{RuleGroup}_{i,j,k}$ for every value $\text{Segment}_{i,j,k}$ within it. $\text{RuleGroup}_{i,j,k}$ contains all rules in RuleSubset_i whose intervals on dimension dim_i cover the value $\text{Segment}_{i,j,k}$. Finally, we use each $\text{Segment}_{i,j}$ in RuleSubset_i as training data to train a corresponding Range-Binarized Model Index ($\text{RBIMI}_{i,j}$), enabling fast localization during the query phase.

During the query phase, we first load the set of incoming packets and perform a sequential search for each packet across the four RuleSubsets, aiming to find the rule that both covers the packet and has the highest priority. For each RuleSubset_i, we invoke up to M pre-trained RBIMI models in parallel, using the packet's value in the dimension dim_i as the query input. Each $\text{RBIMI}_{i,j}$ model returns an index k, which corresponds to its associated candidate rule group $\text{RuleGroup}_{i,j,k}$. We then perform a sequential scan over this rule group to identify the rule that matches the packet and has the highest priority, and record it as the match for the current subset. After searching all four RuleSubsets, we compare the recorded rules and return the rule with the highest global priority as the final match for the packet.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

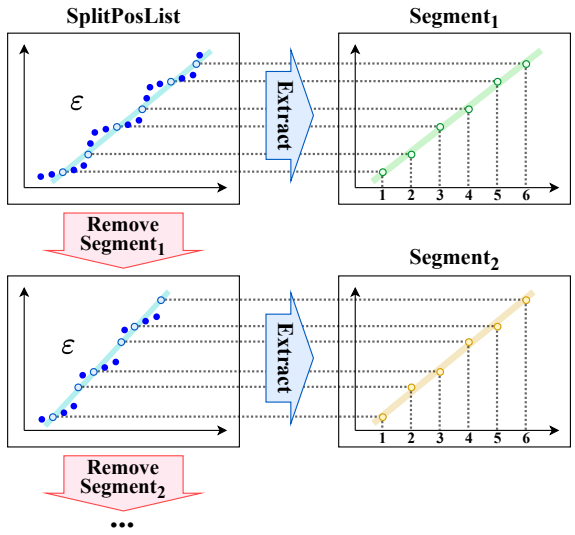


Fig. 6: An example of Distribution-Distance Partitioning.

B. Distribution-Distance Partitioning

Motivation. We observe that a key reason for large model errors and structural redundancy in existing partitioning methods is the lack of an effective measure to characterize the distributional trend of keys during segmentation. Without such a measure, it is difficult to determine whether a segment is suitable for accurate approximation using a single linear model. To address this issue, we propose a greedy partitioning approach based on distributional distance, which evaluates the linear fitting error of key-position pairs in a point-wise manner. By adaptively grouping keys with similar local trends into the same segment, our method improves the accuracy of linear approximations and enables efficient support for parallelizable lookup structures.

Workflow. We propose a greedy global partitioning method based on distributional distance to divide the sequence into multiple segments, each approximated by a linear model. The detailed workflow consists of the following main stages.

(1) *Data Preparation.* We begin by transforming the original SplitPosList into a sequence of points. Each point represents a value and its corresponding index in the original list. That is, the ith point in the sequence should be $(i, spList_i)$. This transformation facilitates a unified treatment in subsequent linear fitting and segment partitioning, contributing to precise segmentation.

(2) *Least Squares Fitting.* For the sequence of points, we apply least squares regression to obtain a fitted line that reflects the linear trend of the data. Specifically, given n points $\{(x_i, y_i)\}_{i=0}^{n-1}$, we compute the slope and intercept as:

$$\hat{b} = \frac{\sum_{i=0}^{n-1}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=0}^{n-1}(x_i - \bar{x})^2} \tag{1}$$

$$\hat{a} = \bar{y} - \hat{b} \cdot \bar{x} \tag{2}$$

The resulting line $y = \hat{a} + \hat{b}x$ is used as the local approximation model for each candidate segment.

TABLE I: Notation of Symbols

Symbol	Description
$spList$	The original SplitPosList to be partitioned.
$sgList$	The final output list of fitted segments.
n	The number of points in the current segment.
M	The maximum allowed number of segments.
$\hat{a}, \hat{b}$	The intercept and slope of the fitted line.
$\bar{x}, \bar{y}$	The average of all x_i and y_i values.
$\mathrm{fix}_Y(y)$	The normalized y-coordinate within the segment.
$\mathrm{dis}(x, y)$	The geometric distance from a point to the fitted line.
$\varepsilon(x, y)$	The distribution distance function for a point.
$\varepsilon_{\max}$	The maximum allowed fitting error per segment.
$hitSpList$	The temporary list of valid points within the error bound.
$resSpList$	The temporary list of points to be processed next.
$\lvert S\rvert$	The total number of constructed segments.

(3) *Distribution Distance Estimation.* To evaluate the fitting quality, we normalize the y-coordinates to a unified scale:

$$\mathrm{fix}_Y(y) = \frac{y - y_{\min} + 1}{y_{\max} - y_{\min} + 1} \cdot n \tag{3}$$

Then we compute the geometric distance from each point to the fitted line:

$$\mathrm{dis}(x, y) = \frac{\left|\hat{b}x - y + \hat{a}\right|}{\sqrt{\hat{b}^2 + 1}} \tag{4}$$

Finally, the distribution distance for a point is calculated as:

$$\varepsilon(x, y) = \mathrm{dis}\big(x, \mathrm{fix}_Y(y)\big)^2 \tag{5}$$

Smaller values of ε indicate stronger linearity and better approximation performance.

(4) *Greedy Segment Construction.* Based on the evaluated fitting quality, we adopt a greedy strategy to construct the final set of segments. At each step, the algorithm selects the longest consecutive segment that satisfies the error constraint. This strategy ensures fitting accuracy while effectively controlling the number of segments.

As shown in Figure 6, each blue dot represents a pair of index and value in the original SplitPosList. Segment 1 is first extracted based on its least-squares fit and error threshold. The points forming Segment 1 are then removed, and the residual points are re-fitted to extract Segment 2, and so on. Each segment ensures an accurate local approximation, and the final segment list captures both smooth trends and sharp transitions in the rule distribution.

Input. The input consists of a monotonically increasing sequence $spList$, an upper bound on the maximum number of segments M, and a tolerance threshold for the fitting error $\varepsilon_{\max}$, such as 10^{-7}.

Objective. The objective is to partition the original sequence into no more than M consecutive segments, where each segment achieves a sufficient linear approximability. Specifically, we aim to control the maximum average distribution distance across all segments within $\varepsilon_{\max}$ as much as possible:

$$\max_{|S| \le M} \sum_{t=1}^{|S|} \bar{\varepsilon}_t, \tag{6}$$

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Greedy-based Distribution-Distance Partitioning

Input: monotonically increasing sequence $spList$; maximum segment number M; threshold $\varepsilon_{\max}$.
Output: segment list $sgList$.

1 let $sgList \leftarrow \emptyset$
Divide up to $M-1$ segments
2 **while** length$(sgList) < M-1$ **do**
3 let $hitSpList \leftarrow \emptyset$
4 let $resSpList \leftarrow \emptyset$
Extract qualified points
5 **for** $i = 0$ **to** length$(spList) - 1$ **do**
6 **if** $\varepsilon(i, spList_i) \leq \varepsilon_{\max}$ **then**
7 Append $spList_i$ into $hitSpList$
8 **else**
9 Append $spList_i$ into $resSpList$
10 Append $hitSpList$ into $sgList$
11 $spList \leftarrow resSpList$
Take remaining points as the last segment
12 **if** *length*$(spList) \neq 0$ **then**
13 Append $spList$ into $sgList$
14 **return** $sgList$

where $\bar{\varepsilon}_t$ denotes the average distribution distance of segment S_t, and $|S|$ is the total number of segments. This formulation improves overall system accuracy while maintaining high efficiency. Furthermore, the resulting segments can be used to construct multiple parallel processing modules, thereby enhancing system throughput.

Constraint 1: Maximum Fitting Error. To ensure that the linear approximation remains accurate for each data point, we require that the distribution distance ε of each point stays below a predefined threshold $\varepsilon_{\max}$, which is set to 10^{-7}:

$$\varepsilon \leq \varepsilon_{\max}. \tag{7}$$

This constraint prevents poor-fitting segments that could degrade overall prediction quality.

Constraint 2: Segment Quantity Limit. To avoid excessive partitioning and maintain system efficiency, we cap the total number of segments by a user-defined upper bound:

$$|S| \leq M, \tag{8}$$

where $|S|$ is the total number of segments and M is the preset maximum.

Solution. We propose a greedy segmentation algorithm using distribution distance. The algorithm 1 describes the process.

First, initialize $sgList$ as an empty sequence(line 1). Then, check whether the length of $sgList$ is less than $M-1$, if so, enter the loop(line 2). Next, filter out $spList$ whose distribution distance ε is not greater than $\varepsilon_{\max}$, store them in $hitSpList$, and save the unselected values in $resSpList$(lines 3 to 9). After that, take the sequence $hitSpList$ as an element and append it to the end of $sgList$(line 10), and update $spList$ to $resSpList$ for the next iteration(line 11). Once the loop ends, if $spList$ is not empty, append it to the end of $sgList$(lines 12 to 13). The final $sgList$ is the desired result(line 14). The length of $sgList$ will not exceed M.

C. Base-Index Representation

To overcome the inefficiency of floating-point operations and achieve hardware-friendly model evaluation, EffiMatch introduces a *Base-Index Representation (BI)*. This novel encoding strategy accurately emulates floating-point linear models through integer-only computations. In this section, we elaborate on the core design insights, operational workflow, and theoretical accuracy guarantees of BI.

Insight. The key challenge addressed by BI is to support high-precision linear evaluation without relying on floating-point units (FPUs). Rather than using fixed global scaling, BI allows each model coefficient to carry its shift magnitude, thereby preserving dynamic range and precision locally. Specifically, any real value v is encoded as:

$$v = \mathrm{BI}(v_{\text{base}}, v_{\text{index}}) = v_{\text{base}} \times 2^{v_{\text{index}}} \tag{9}$$

where v_{base} and v_{index} are signed 64-bit integers. This design is inspired by the exponent-mantissa separation in IEEE-754 floating-point representations, but it adopts a simpler design that effectively eliminates the reliance on floating-point arithmetic: v_{base} captures the coarse value, while v_{index} adjusts the scale by bit-shifting.

Since the parameters trained by the RBIMI model are floating-point numbers, they need conversion to BI representation. Here, we extract up to the first 30 significant digits of one's binary form as its base integer, ignore the decimal point, and adjust its index to correct the truth value.

During runtime inference, prealigned shift operations avoid costly normalization or division, guaranteeing deterministic low-latency datapaths. The BI structure thus seamlessly replaces the role of (a, b) in the affine transformation $y = ax+b$, without any floating-point multiplication or rounding artifacts.

Workflow. At inference time, given coefficients $a = \mathrm{BI}(a_{\text{base}}, a_{\text{index}})$ and $b = \mathrm{BI}(b_{\text{base}}, b_{\text{index}})$, and input x, we compute the BI representation of prediction value y by separately scaling and aligning $a \cdot x$ and b according to their indices.

If $a_{\text{index}} > b_{\text{index}}$, we apply:

$$y = \mathrm{BI}\big[(a_{\text{base}} \times x) \ll (a_{\text{index}} - b_{\text{index}}) + b_{\text{base}},\ b_{\text{index}}\big] \tag{10}$$

Else, we apply:

$$y = \mathrm{BI}\big[a_{\text{base}} \times x + \big(b_{\text{base}} \ll (b_{\text{index}} - a_{\text{index}})\big),\ a_{\text{index}}\big] \tag{11}$$

Finally, the above BI representation is transformed into an integer via our rounding function. Figure 7 illustrates the overall workflow of Base-Index Representation. The floating-point parameters a and b obtained during the RBIMI training phase are quantized into the integer-based BI format. Given an integer input x, EffiMatch performs the computation of $y = ax + b$ entirely using integer operations, resulting in a

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

BI representation $(y_{\text{base}}, y_{\text{index}})$. Finally, the predicted value is obtained through BI rounding.

Rounding Strategy. To produce the final integer output from $y = ax+b$ under Base-Index arithmetic, we adopt a rounding-to-nearest strategy. Specifically, for any value y, we know the rounding operation could be defined as:

$$\text{round}(y) = \lfloor y + 0.5 \rfloor \tag{12}$$

When $y_{\text{index}} \geq 0$, the addition of 0.5 does not affect the truncation result:

$$\begin{aligned} \lfloor y + 0.5 \rfloor &= \lfloor y_{\text{base}} \times 2^{y_{\text{index}}} + 0.5 \rfloor \\ &= \lfloor y_{\text{base}} \times 2^{y_{\text{index}}} \rfloor \\ &= y_{\text{base}} \gg y_{\text{index}} \end{aligned} \tag{13}$$

On the contrary, when $y_{\text{index}} < 0$, we apply correction by representing 0.5 in the scaled domain:

$$\begin{aligned} \lfloor y + 0.5 \rfloor &= \lfloor y_{\text{base}} \times 2^{y_{\text{index}}} + 2^{-1} \rfloor \\ &= \lfloor \left(y_{\text{base}} + 2^{-y_{\text{index}}-1} \right) \div 2^{-y_{\text{index}}} \rfloor \\ &= \left(y_{\text{base}} + \left(1 \ll (-y_{\text{index}} - 1) \right) \right) \gg -y_{\text{index}} \end{aligned} \tag{14}$$

We define two rounding cases as follows:

$$r_1 = y_{\text{base}} \gg y_{\text{index}} \tag{15}$$

$$r_2 = \left(y_{\text{base}} + \left(1 \ll (-y_{\text{index}} - 1) \right) \right) \gg -y_{\text{index}} \tag{16}$$

Therefore, the rounding operation can be defined as:

$$\text{round}(y) = \begin{cases} r_1, & y_{\text{index}} \geq 0 \\ r_2, & \text{otherwise} \end{cases} \tag{17}$$

Example. Consider a real-valued affine function with coefficients $a = 1.5625$ and $b = 0.625$, and an input value $x = 6$. The true output is calculated as:

$$y = a \times x + b = 1.5625 \times 6 + 0.625 = 10.0 \tag{18}$$

$$\text{round}(y) = 10 \tag{19}$$

To apply BI representation, a and b are quantized separately. We represent $a = \text{BI}(25, -4)$, since $1.5625 = 25 \times 2^{-4}$. Similarly, $b = \text{BI}(5, -3)$, since $0.625 = 5 \times 2^{-3}$. At inference time, we first compute $a_{\text{base}} \times x = 25 \times 6 = 150$. Since $a_{\text{index}} = -4$ is smaller than $b_{\text{index}} = -3$, we shift b_{base} left by $(-3) - (-4) = 1$ bit, yielding $5 \ll 1 = 10$, and then add to 150 to obtain 160. Then we can represent $y = \text{BI}(160, -4)$. In the rounding phase, since $y_{\text{index}} = -4 < 0$, we add $1 \ll (4 - 1) = 8$ to y_{base} to get 168. Finally, we get $\text{round}(y) = 168 \gg 4 = 10$. Thus, using only integer shift, addition, and multiplication, we accurately reconstruct the original output.

Analysis. At the stage of converting floating-point value v to BI representation, the error of value v caused by intercepting 30 bits of binary significant digits is bounded by:

$$\varepsilon_v = \left| v - \frac{\lfloor v \cdot 2^{30} \rfloor}{2^{30}} \right| = \frac{\left| v \cdot 2^{30} - \lfloor v \cdot 2^{30} \rfloor \right|}{2^{30}} \leq 2^{-30} \tag{20}$$

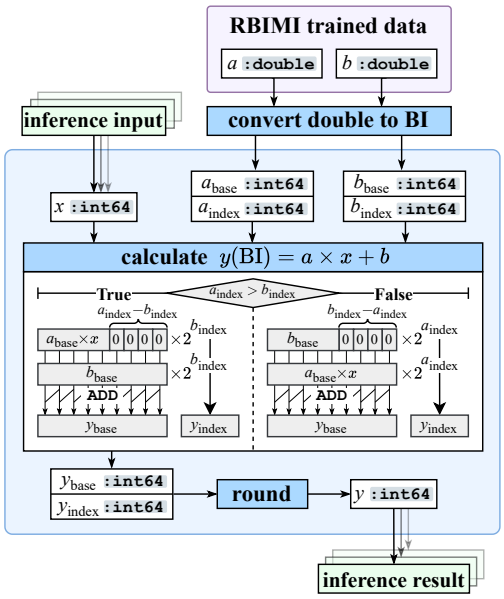


Fig. 7: An example of Base-Index Representation.

Let $y = ax + b$ denote the truth output, and let $\hat{y}$ denote the output produced by BI arithmetic. The absolute error between $\hat{y}$ and y is bounded by:

$$|\hat{y} - y| = |x| \cdot \varepsilon_a + \varepsilon_b \leq (|x| + 1) \cdot 2^{-30} \tag{21}$$

Moreover, the computational complexity of BI inference is highly efficient: Each evaluation requires only a single integer multiplication, two integer additions, and two bit-shift operations. Since all operations are deterministic and bounded, the inference latency is constant and predictable, independent of input magnitude. This characteristic significantly accelerates the inference speed of the BI format.

IV. EVALUATION

Testbed. We ran the simulation experiments on a 13th Gen Intel(R) Core(TM) i9-13900H processor within a WSL2 (Ubuntu 22.04) environment. The system was configured with 20 assigned CPU cores, 16 GB of main memory, and 4 GB of swap space, without utilizing any GPU acceleration. All experiments were conducted entirely on the CPU.

Rule Set. To construct evaluation workloads, we adopted the ClassBench [32] benchmark suite, which provides standardized rule sets and corresponding packet traces. The rule sets are categorized into three typical types: access control lists (acl1–acl5), firewalls (fw1–fw5), and IP chains (ipc1, ipc2). We generated rule sets at three different scales: 1k, 10k, and 100k entries. For packet testing, we produced packet traces consisting of 500k entries to ensure statistical significance.

Comparison solutions. We compare EffiMatch with six representative state-of-the-art packet classification approaches:

- NuevoMatch [14]: the first learned packet classification (LPC) method that leverages shallow neural models with compact memory usage under large-scale rule sets.
- NeuTree [15]: a recent LPC framework optimized for FPGA deployment, which replaces floating-point operations with

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

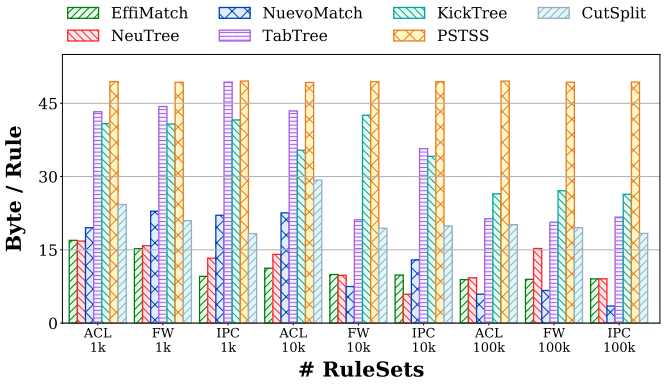


Fig. 8: (Exp#1) Memory consumption of classifiers.

binarized logic to enable resource-efficient and low-latency inference on hardware.

- CutSplit [26]: a classical decision-tree-based algorithm that combines Cut and Split heuristics to minimize rule replication and achieve high performance.
- TabTree [27]: a hardware-friendly decision tree method that reduces memory usage and accelerates classification by performing efficient bit-level partitioning.
- KickTree [33]: an enhancement of TabTree that supports kick-based partitioning and enables rule-copy-free tree construction for better scalability.
- PSTSS [25]: a traditional packet classification algorithm implemented in OpenVSwitch, known for its stable memory usage and throughput consistency.

A. Memory Consumption

(Exp#1) Memory consumption of classifiers. We evaluate the memory consumption of classifiers by measuring the average bytes per rule (Byte/Rule), as shown in Figure 8. EffiMatch achieves significantly lower memory consumption than existing methods across ACL, FW, and IPC rule sets at various scales (1k, 10k, and 100k). Specifically, compared to the learned index method *NuevoMatch*, EffiMatch reduces memory usage by approximately 19.35%, primarily benefiting from its Distribution-Distance Partitioning (DDP) strategy, which partitions rules based on key distribution and distance constraints, effectively minimizing storage requirements. Moreover, compared to traditional decision-tree methods such as *CutSplit*, *TabTree*, *KickTree*, and *PSTSS*, EffiMatch achieves memory savings of up to 68.37%. Traditional methods typically rely on large decision trees and frequent rule replication to maintain classification correctness, leading to increased memory overhead. In contrast, EffiMatch eliminates the need for rule replication and deep tree structures, fundamentally reducing memory usage during classification.

B. Construction Efficiency

(Exp#2) Construction Time of LPC methods. We evaluate the construction time of EffiMatch against *NeuTree* and *NuevoMatch* under rule sets of different sizes (1k, 10k, and 100k), as shown in Figure 9. Compared to *NuevoMatch*, EffiMatch achieves a 2–4 orders of magnitude reduction in

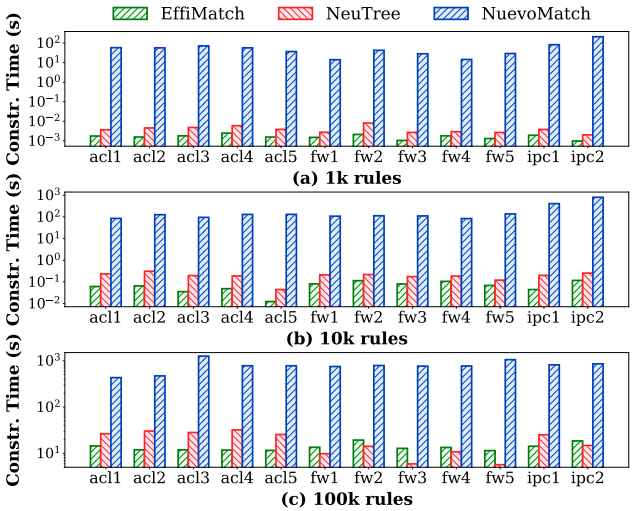


Fig. 9: (Exp#2) Construction Time of LPC methods.

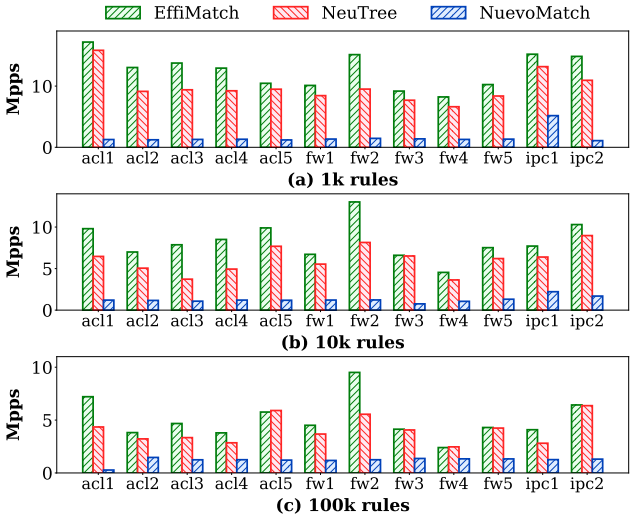


Fig. 10: (Exp#3) Lookup Throughput.

construction time, benefiting from its lightweight Distribution-Distance Partitioning (DDP), which avoids complex neural network training and reduces the overhead of RBIMI fitting. Compared to *NeuTree*, EffiMatch achieves shorter construction time, with reductions of 55.67% on 1k rules, 63.56% on 10k rules, and 27.08% on 100k rules. The advantage becomes more evident as rule size increases, showing better scalability and faster index construction without bucket partitioning or iterative tuning. These properties make EffiMatch highly suitable for large-scale packet classification tasks.

C. Lookup Performance

(Exp#3) Lookup Throughput. We evaluate the lookup throughput (Mpps) of EffiMatch compared with *NeuTree* and *NuevoMatch* under varying rule set sizes (1k, 10k, and 100k), as illustrated in Figure 10. Experimental results demonstrate that EffiMatch consistently achieves higher packet processing throughput than *NeuTree* and *NuevoMatch* across all tested

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

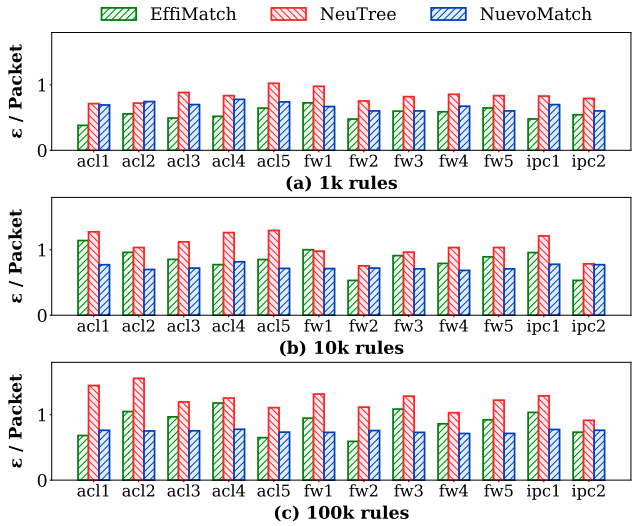


Fig. 11: (Exp#4) Average Offset of Predicted Results.

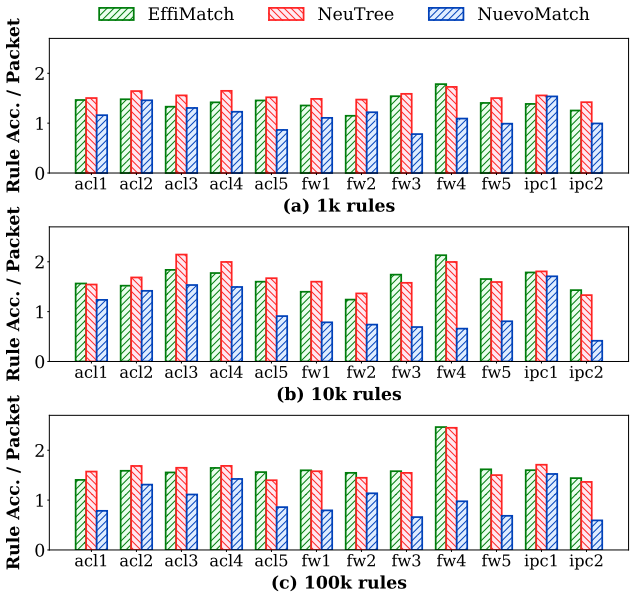


Fig. 12: (Exp#5) Rule Access Efficiency.

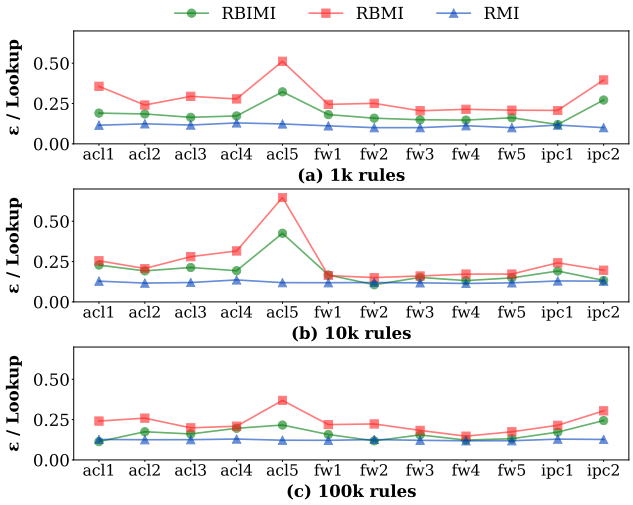


Fig. 13: (Exp#6) Single Lookup Offset Analysis.

scenarios. Specifically, EffiMatch increases throughput by approximately 30.71% on average compared to *NeuTree*, and by up to astonishing 6 times compared to *NuevoMatch*. This advantage primarily results from EffiMatch's parallel lookup mechanism of the RBIMI model, which makes M (usually 4 or 8, depending on the number of CPU cores) lookups available at the same time, and also from Distribution-Distance Partitioning (DDP) and Base-Index (BI) Representation, which effectively reduce the error range during lookups, decreasing unnecessary traversal.

(Exp#4) Average Offset of Predicted Results. We evaluate the average offset per packet of EffiMatch compared with *NeuTree* and *NuevoMatch* under varying rule set sizes (1k, 10k, and 100k), as shown in Figure 11. The experimental results indicate that EffiMatch consistently achieves lower prediction offsets than *NeuTree*, with reductions of approximately 33.80% for 1k rules, 19.82% for 10k rules, and 26.90% for 100k rules. Compared to *NuevoMatch*, EffiMatch reduces the offset by approximately 17.10% at the 1k rule set, while exhibiting similar offsets at 10k and 100k rule sets. This performance improvement primarily results from EffiMatch's reduction of prediction errors during rule lookups, thereby enhancing classification accuracy.

(Exp#5) Rule Access Efficiency. We evaluate the average number of rule accesses per packet for EffiMatch under rule sets of different sizes (1k, 10k, and 100k), as shown in Figure 12. Compared with *NeuTree*, EffiMatch reduces rule access by approximately 8.77% in the 1k rule set, 2.71% in the 10k rule set, and achieves nearly identical results at 100k. Compared to *NuevoMatch*, EffiMatch exhibits slightly higher or comparable rule accesses in certain scenarios. This is primarily because *NuevoMatch* employs a floating-point neural predictor to guide efficient lookups, whereas EffiMatch uses a fully float-free Base-Index structure, balancing speed and accuracy. Despite the absence of neural guidance, EffiMatch demonstrates competitive lookup efficiency through its lightweight design.

(Exp#6) Single Lookup Offset Analysis. We evaluate the average offset per lookup of RBIMI, RBMI, and RMI under different rule set sizes (1k, 10k, and 100k), as shown in Figure 13. Compared to RBMI, RBIMI reduces the offset by approximately 33.80%, 19.82%, and 26.90% , benefiting from a effective rule partitioning strategy (DDP) and a compact Base-Index (BI) structure for precise segment-wise modeling. Although RBIMI shows slightly higher offset than RMI due to the lack of floating-point computation and global modeling, it adopts a float-free design that reduces complexity while preserving competitive accuracy. This trade-off enables faster inference and suitability for resource-constrained deployments, reflecting EffiMatch's balanced design between precision and efficiency.

(Exp#7) Robustness Under Skewed Rule Distributions. To evaluate EffiMatch's robustness under non-uniform rule distributions, we construct three synthetic rule sets, illustrated in

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

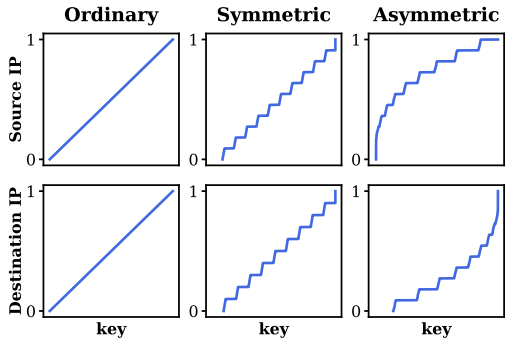


Fig. 14: (Exp#7) Distribution of Source and Destination IPs in Constructed Non-Uniform Rule Sets.

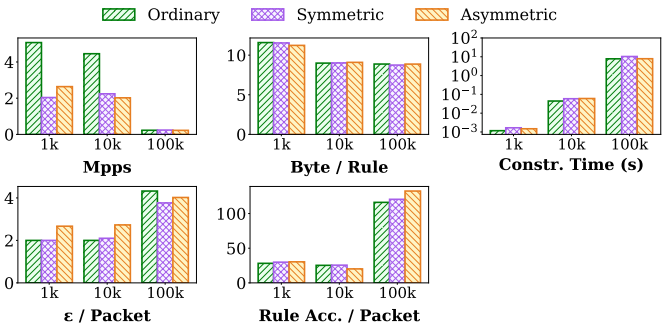


Fig. 15: (Exp#7) Robustness Under Skewed Rule Distributions.

Figure 14: *Ordinary*, where source and destination IPs follow an approximately linear distribution; *Symmetric*, where both follow the same highly skewed distribution; and *Asymmetric*, where source and destination IPs follow opposite skewed distributions to induce maximal imbalance. Each rule set is generated at 1k, 10k, and 100k scales, with five rule sets and 400,000 trace entries per category.

We evaluate five key metrics: throughput (Mpps), memory usage (Byte/Rule), construction time (s), and average error and rule accesses per packet. As shown in Figure 15, throughput moderately declines under extreme skew due to increased segmentation and parallel overhead, which result from more complex query structures. This complexity arises as a tradeoff to maintain high lookup rates under highly skewed rule layouts. Despite this, memory and construction time remain stable. Error and rule access also stay bounded, indicating that DDP partitioning and BI encoding preserve inference quality under complex layouts. These results confirm EffiMatch's robustness and scalability across imbalanced traffic scenarios.

V. RELATED WORK

Packet classification algorithms. Existing packet classification algorithms can be broadly divided into three categories: hash-based, decomposition-based, and decision tree-based methods. Hash-based approaches, such as Tuple Space Search [25], group rules by prefix lengths and perform parallel lookups, but their performance can vary significantly across different rule sets. Decomposition-based methods, such as Cross-producting [21], split multi-dimensional rules into lookup units and merge the results, performing well on small rule sets but leading to high memory usage at large scales. Decision tree-based methods, such as HyperCuts [34] and EffiCuts [35], adaptively partition the space to reduce lookup complexity but usually require costly preprocessing.

Learned Index Structures. With the rise of deep learning and hardware acceleration, learned indexes are gaining traction in database systems as alternatives to traditional structures like B-trees and hash tables. RMI [11] pioneered this direction by approximating the cumulative distribution function (CDF) of keys to predict storage locations, greatly reducing query latency and memory consumption. Systems like ALEX [36], Flood [37], and Limousine [38] extend learned indexes for dynamic workloads and integrate them with classical structures for distribution-aware I/O optimization, enabling automatic adaptation across diverse system configurations.

VI. DISCUSSION

Online Rule Update. Although EffiMatch exhibits strong inference efficiency and hardware compatibility, its current design is primarily oriented toward static rule sets. For scenarios requiring online rule updates, we propose a two-tier mechanism with bounded latency. The first tier is an updatable small-scale front end (e.g., TCAM or fast rule buffer) that temporarily stores newly inserted or modified rules and matches them before the EffiMatch model. The second tier periodically merges these changes into the model at the segment level: since DDP (§III-B) partitions the key space, each update affects only one segment. During maintenance, queries use the previous version, and an atomic pointer swap ensures seamless transition after reconstruction. With BI's (§III-C) integer-only linear model, these operations are deterministic, lightweight, and avoid full retraining. Although a comprehensive exploration of update policies (e.g., trigger conditions, concurrency handling) is beyond the scope of this paper, this design provides a practical path for online rule updates and remains a key direction for future work.

VII. ACKNOWLEDGMENT

We thank the reviewers for their constructive comments. Dong Zhang, Hongyan Liu, and Jiashuo Yu are the corresponding authors. This work was supported by the National Key R&D Program of China (2023YFB2904000).

VIII. CONCLUSION

This paper introduces EffiMatch, a learning-based packet classification architecture that addresses the trade-off between RMI complexity and linear search range. It presents two key designs: Distribution-Distance Partitioning (DDP), which segments rules by trend similarity to reduce search range with parallel lookup, and Base-Index Representation (BI), which replaces floating-point weights with expressive integer formats without increasing model complexity. EffiMatch employs a parallel multi-model lookup structure and outperforms state-of-the-art learning-based methods in search range, lookup speed, and model construction efficiency.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] C. Jia, C. Li, Y. Li, X. Hu, and J. Li, "An observation of packet classification: Most rules are at the top," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2022, pp. 1–6.

[2] C. Jung, S. Kim, R. Jang, D. Mohaisen, and D. Nyang, "A scalable and dynamic acl system for in-network defense," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1679–1693.

[3] R. MacDavid, C. Cascone, P. Lin, B. Padmanabhan, A. Thakur, L. Peterson, J. Rexford, and O. Sunay, "A p4-based 5g user plane function," in *Proceedings of the ACM SIGCOMM Symposium on SDN Research (SOSR)*, 2021, pp. 162–168.

[4] M. Dalton, D. Schultz, J. Adriaens, A. Arefin, A. Gupta, B. Fahs, D. Rubinstein, E. C. Zermeno, E. Rubow, J. A. Docauer *et al.*, "Andromeda: Performance, isolation, and velocity at scale in cloud network virtualization," in *15th USENIX symposium on networked systems design and implementation (NSDI 18)*, 2018, pp. 373–387.

[5] X.-N. Nguyen, D. Saucez, C. Barakat, and T. Turletti, "Rules placement problem in openflow networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1273–1286, 2015.

[6] S. Palkar, C. Lan, S. Han, K. Jang, A. Panda, S. Ratnasamy, L. Rizzo, and S. Shenker, "E2: A framework for nfv applications," in *Proceedings of the 25th Symposium on Operating Systems Principles*, 2015, pp. 121–136.

[7] G. P. Katsikas, T. Barbette, D. Kostic, R. Steinert, and G. Q. Maguire Jr, "Metron:{NFV} service chains at the true speed of the underlying hardware," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 171–186.

[8] D. Firestone, A. Putnam, S. Mundkur, D. Chiou, A. Dabagh, M. Andrewartha, H. Angepat, V. Bhanu, A. Caulfield, E. Chung *et al.*, "Azure accelerated networking:{SmartNICs} in the public cloud," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 51–66.

[9] Y. Feng, Z. Chen, H. Song, Y. Zhang, H. Zhou, R. Sun, W. Dong, P. Lu, S. Liu, C. Zhang *et al.*, "Empower programmable pipeline for advanced stateful packet processing," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 491–508.

[10] P. Bosshart, G. Gibb, H.-S. Kim, G. Varghese, N. McKeown, M. Izzard, F. Mujica, and M. Horowitz, "Forwarding metamorphosis: Fast programmable match-action processing in hardware for sdn," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 99–110, 2013.

[11] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proceedings of the 2018 international conference on management of data*, 2018, pp. 489–504.

[12] A. Rashelbach, O. Rottenstreich, and M. Silberstein, "Scaling open {vSwitch} with a computational cache," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 1359–1374.

[13] B. Pfaff, J. Pettit, T. Koponen, E. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar *et al.*, "The design and implementation of open {vSwitch}," in *12th USENIX symposium on networked systems design and implementation (NSDI 15)*, 2015, pp. 117–130.

[14] A. Rashelbach, O. Rottenstreich, and M. Silberstein, "A computational approach to packet classification," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 542–556.

[15] J. Yu, L. Zhu, L. Huang, X. Chen, L. Zheng, D. Zhang, X. Chen, and C. Wu, "Scaling learning-based packet classification hardware with neutree," in *Proceedings of IEEE INFOCOM*, 2025, to appear.

[16] Intel, "Barefoot tofino," https://www.barefootnetworks.com/technology/\#tofino, accessed: YYYY-MM-DD.

[17] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1," *arXiv preprint arXiv:1602.02830*, 2016.

[18] Y. Zhu, N. Kang, J. Cao, A. Greenberg, G. Lu, R. Mahajan, D. Maltz, L. Yuan, M. Zhang, B. Y. Zhao *et al.*, "Packet-level telemetry in large datacenter networks," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, 2015, pp. 479–491.

[19] S. Zhao, M. Chandrashekar, Y. Lee, and D. Medhi, "Real-time network anomaly detection system using machine learning," in *2015 11th international conference on the design of reliable communication networks (drcn)*. IEEE, 2015, pp. 267–270.

[20] C. Cranor, T. Johnson, O. Spataschek, and V. Shkapenyuk, "Gigascope: A stream database for network applications," in *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, 2003, pp. 647–651.

[21] P. Gupta and N. McKeown, "Packet classification using hierarchical intelligent cuttings," in *Hot Interconnects VII*, vol. 40, 1999.

[22] Y. Qi, L. Xu, B. Yang, Y. Xue, and J. Li, "Packet classification algorithms: From theory to practice," in *IEEE INFOCOM 2009*. IEEE, 2009, pp. 648–656.

[23] Z. Liu, S. Sun, H. Zhu, J. Gao, and J. Li, "Bitcuts: A fast packet classification algorithm using bit-level cutting," *Computer Communications*, vol. 109, pp. 38–52, 2017.

[24] J. Daly, V. Bruschi, L. Linguaglossa, S. Pontarelli, D. Rossi, J. Tollet, E. Torng, and A. Yourtchenko, "Tuplemerge: Fast software packet processing for online packet classification," *IEEE/ACM transactions on networking*, vol. 27, no. 4, pp. 1417–1431, 2019.

[25] V. Srinivasan, S. Suri, and G. Varghese, "Packet classification using tuple space search," in *Proceedings of the conference on Applications, technologies, architectures, and protocols for computer communication*, 1999, pp. 135–146.

[26] W. Li, X. Li, H. Li, and G. Xie, "Cutsplit: A decision-tree combining cutting and splitting for scalable packet classification," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 2645–2653.

[27] W. Li, T. Yang, Y.-K. Chang, T. Li, and H. Li, "Tabtree: A tss-assisted bit-selecting tree scheme for packet classification with balanced rule mapping," in *2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. IEEE, 2019, pp. 1–8.

[28] W. Li, T. Yang, O. Rottenstreich, X. Li, G. Xie, H. Li, B. Vamanan, D. Li, and H. Lin, "Tuple space assisted packet classification with high performance on both search and update," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 7, pp. 1555–1569, 2020.

[29] Y. Xin, W. Li, G. Xie, Y. Xu, and Y. Wang, "Updatable packet classification on fpga with bounded worst-case performance," in *2022 IEEE Symposium on High-Performance Interconnects (HOTI)*. IEEE, 2022, pp. 21–28.

[30] Y. Xin, W. Li, G. Tang, T. Yang, X. Hu, and Y. Wang, "Fpga-based updatable packet classification using tss-combined bit-selecting tree," *IEEE/ACM Transactions on Networking*, vol. 30, no. 6, pp. 2760–2775, 2022.

[31] N. Zilberman, Y. Audzevich, G. A. Covington, and A. W. Moore, "Netfpga sume: Toward 100 gbps as research commodity," *IEEE micro*, vol. 34, no. 5, pp. 32–41, 2014.

[32] D. E. Taylor and J. S. Turner, "Classbench: A packet classi-

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.

fication benchmark," *IEEE/ACM transactions on networking*, vol. 15, no. 3, pp. 499–511, 2007.
[33] Y. Xin, W. Li, C. Jia, X. Li, Y. Xu, B. Liu, Z. Tian, and W. Zhang, "Recursive multi-tree construction with efficient rule sifting for packet classification on fpga," *IEEE/ACM Transactions on Networking*, vol. 32, no. 2, pp. 1707–1722, 2023.
[34] S. Singh, F. Baboescu, G. Varghese, and J. Wang, "Packet classification using multidimensional cutting," in *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications*, 2003, pp. 213–224.
[35] B. Vamanan, G. Voskuilen, and T. Vijaykumar, "Efficuts: Optimizing packet classification for memory and throughput," *ACM SIGCOMM Computer Communication Review*, vol. 40, no. 4, pp. 207–218, 2010.
[36] J. Ding, U. F. Minhas, J. Yu, C. Wang, J. Do, Y. Li, H. Zhang, B. Chandramouli, J. Gehrke, D. Kossmann *et al.*, "Alex: an updatable adaptive learned index," in *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, 2020, pp. 969–984.
[37] V. Nathan, J. Ding, M. Alizadeh, and T. Kraska, "Learning multi-dimensional indexes," in *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, 2020, pp. 985–1000.
[38] S. Chatterjee, M. F. Pekala, L. Kruglyak, and S. Idreos, "Limousine: Blending learned and classical indexes to self-design larger-than-memory cloud storage engines," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–28, 2024.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:23 UTC from IEEE Xplore. Restrictions apply.